Software Engineering - GenAI Project Report

Caoimhe Coveney McKeown
25249131

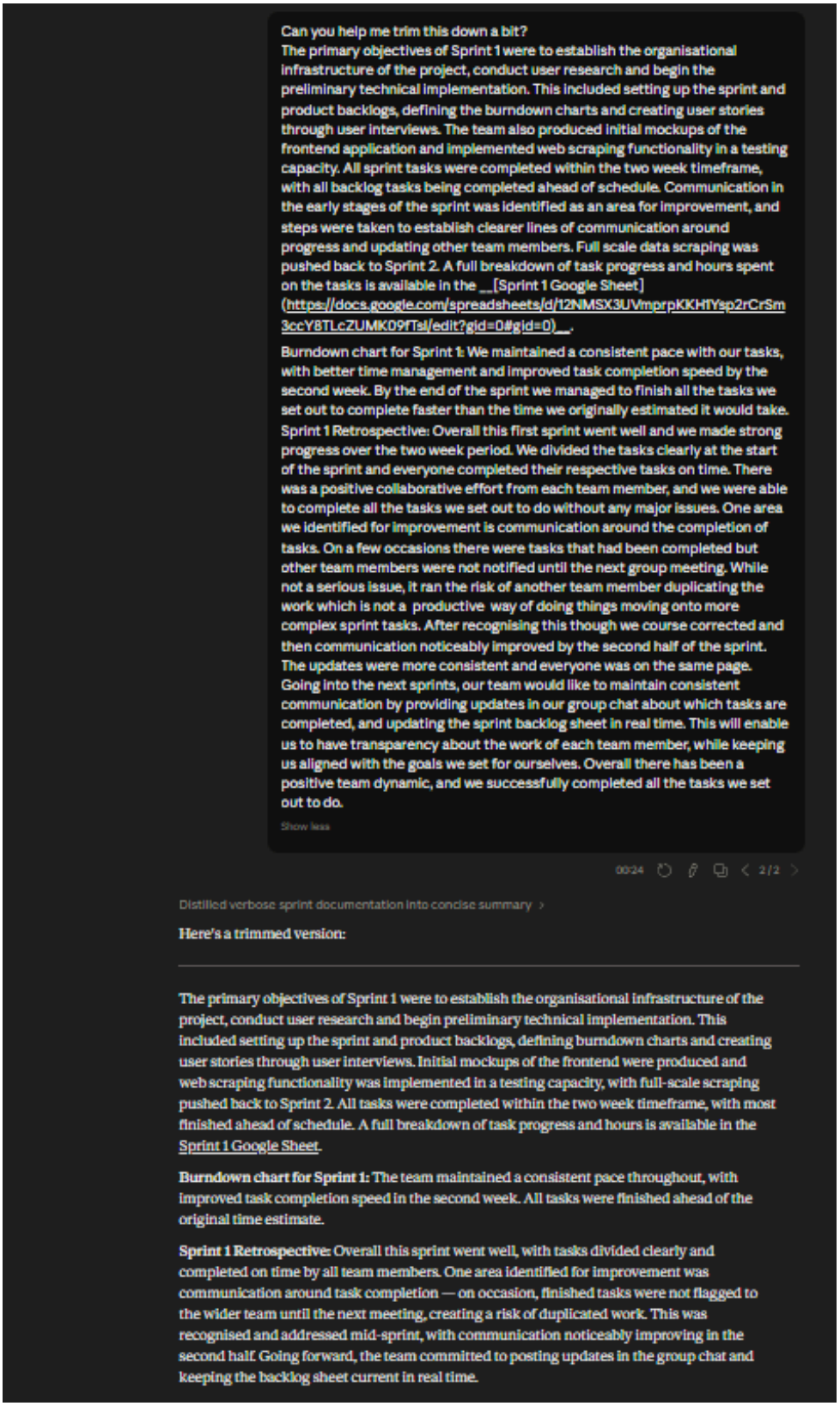
GenAI was used to support the development of the project report, primarily for refining and condensing written content. Claude AI was specifically used to assist in reducing sections of the report to meet page limits. The original content was written by me, and any AI-assisted revisions were used to improve clarity and conciseness while preserving the intended meaning.

GenAI was also used for generating design mockups and diagrams via Figma AI tools; these are documented separately in the corresponding GenAI design file and were incorporated into the final report.

Content of chats:

- Advice on reducing the length of the sprint sections
- Guidance on structuring report sections
- Refining writing content for clarity and conciseness

Sprint 1:



Can you help me trim this down a bit?
The primary objectives of Sprint 1 were to establish the organisational infrastructure of the project, conduct user research and begin the preliminary technical implementation. This included setting up the sprint and product backlogs, defining the burndown charts and creating user stories through user interviews. The team also produced initial mockups of the frontend application and implemented web scraping functionality in a testing capacity. All sprint tasks were completed within the two week timeframe, with all backlog tasks being completed ahead of schedule. Communication in the early stages of the sprint was identified as an area for improvement, and steps were taken to establish clearer lines of communication around progress and updating other team members. Full scale data scraping was pushed back to Sprint 2. A full breakdown of task progress and hours spent on the tasks is available in the __[Sprint 1 Google Sheet](https://docs.google.com/spreadsheets/d/12NMSX3UVmprpKKHtYsp2rCrSm3ccY8TLcZUMK09fTsI/edit?gid=0#gid=0)__.

Burndown chart for Sprint 1: We maintained a consistent pace with our tasks, with better time management and improved task completion speed by the second week. By the end of the sprint we managed to finish all the tasks we set out to complete faster than the time we originally estimated it would take. Sprint 1 Retrospective: Overall this first sprint went well and we made strong progress over the two week period. We divided the tasks clearly at the start of the sprint and everyone completed their respective tasks on time. There was a positive collaborative effort from each team member, and we were able to complete all the tasks we set out to do without any major issues. One area we identified for improvement is communication around the completion of tasks. On a few occasions there were tasks that had been completed but other team members were not notified until the next group meeting. While not a serious issue, it ran the risk of another team member duplicating the work which is not a productive way of doing things moving onto more complex sprint tasks. After recognising this though we course corrected and then communication noticeably improved by the second half of the sprint. The updates were more consistent and everyone was on the same page. Going into the next sprints, our team would like to maintain consistent communication by providing updates in our group chat about which tasks are completed, and updating the sprint backlog sheet in real time. This will enable us to have transparency about the work of each team member, while keeping us aligned with the goals we set for ourselves. Overall there has been a positive team dynamic, and we successfully completed all the tasks we set out to do.

Show less

00:24 2 / 2

Distilled verbose sprint documentation into concise summary

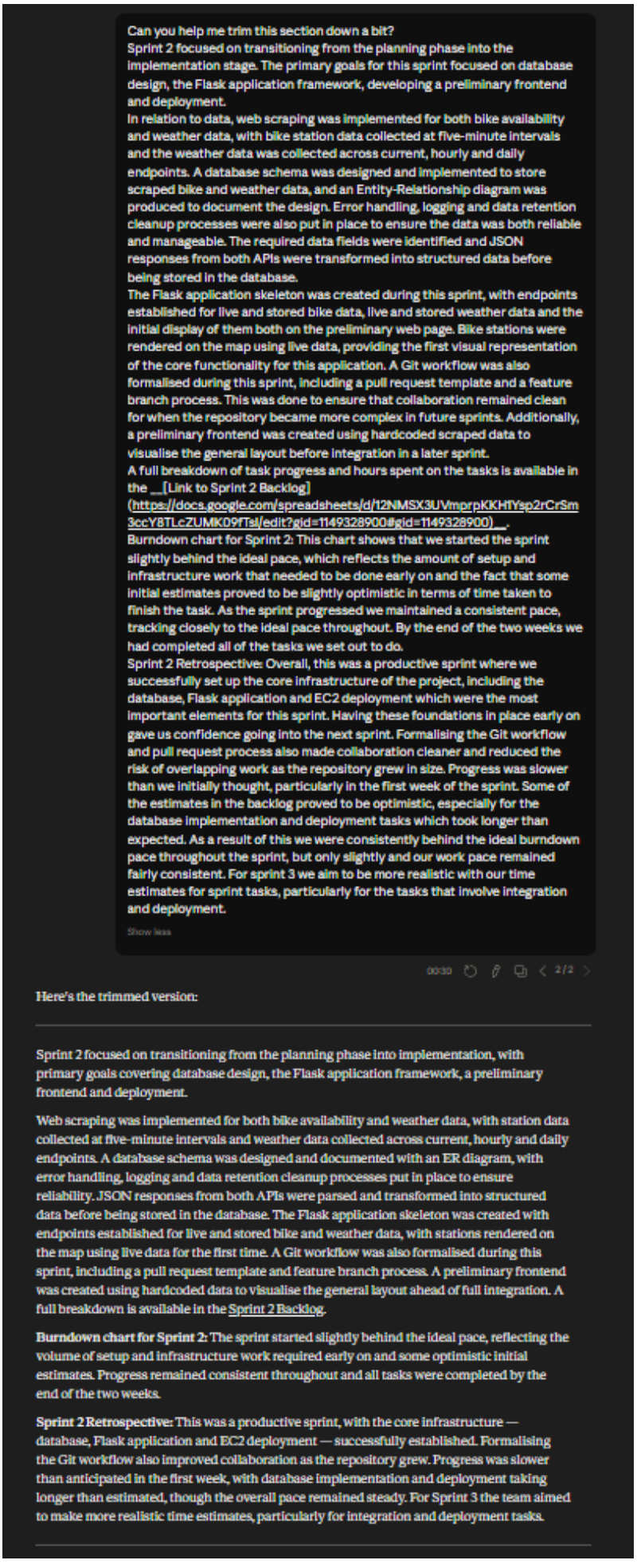
Here's a trimmed version:

The primary objectives of Sprint 1 were to establish the organisational infrastructure of the project, conduct user research and begin preliminary technical implementation. This included setting up the sprint and product backlogs, defining burndown charts and creating user stories through user interviews. Initial mockups of the frontend were produced and web scraping functionality was implemented in a testing capacity, with full-scale scraping pushed back to Sprint 2. All tasks were completed within the two week timeframe, with most finished ahead of schedule. A full breakdown of task progress and hours is available in the Sprint 1 Google Sheet.

Burndown chart for Sprint 1: The team maintained a consistent pace throughout, with improved task completion speed in the second week. All tasks were finished ahead of the original time estimate.

Sprint 1 Retrospective: Overall this sprint went well, with tasks divided clearly and completed on time by all team members. One area identified for improvement was communication around task completion — on occasion, finished tasks were not flagged to the wider team until the next meeting, creating a risk of duplicated work. This was recognised and addressed mid-sprint, with communication noticeably improving in the second half. Going forward, the team committed to posting updates in the group chat and keeping the backlog sheet current in real time.

Sprint 2:



Can you help me trim this section down a bit?
Sprint 2 focused on transitioning from the planning phase into the implementation stage. The primary goals for this sprint focused on database design, the Flask application framework, developing a preliminary frontend and deployment.
In relation to data, web scraping was implemented for both bike availability and weather data, with bike station data collected at five-minute intervals and the weather data was collected across current, hourly and daily endpoints. A database schema was designed and implemented to store scraped bike and weather data, and an Entity-Relationship diagram was produced to document the design. Error handling, logging and data retention cleanup processes were also put in place to ensure the data was both reliable and manageable. The required data fields were identified and JSON responses from both APIs were transformed into structured data before being stored in the database.
The Flask application skeleton was created during this sprint, with endpoints established for live and stored bike data, live and stored weather data and the initial display of them both on the preliminary web page. Bike stations were rendered on the map using live data, providing the first visual representation of the core functionality for this application. A Git workflow was also formalised during this sprint, including a pull request template and a feature branch process. This was done to ensure that collaboration remained clean for when the repository became more complex in future sprints. Additionally, a preliminary frontend was created using hardcoded scraped data to visualise the general layout before integration in a later sprint.
A full breakdown of task progress and hours spent on the tasks is available in the __[Link to Sprint 2 Backlog](https://docs.google.com/spreadsheets/d/12NMSX3UVmprpKKHtYsp2rCrSm3ccY8TLcZUMK09fTsI/edit?gid=1149328900#gid=1149328900)__.
Burndown chart for Sprint 2: This chart shows that we started the sprint slightly behind the ideal pace, which reflects the amount of setup and infrastructure work that needed to be done early on and the fact that some initial estimates proved to be slightly optimistic in terms of time taken to finish the task. As the sprint progressed we maintained a consistent pace, tracking closely to the ideal pace throughout. By the end of the two weeks we had completed all of the tasks we set out to do.
Sprint 2 Retrospective: Overall, this was a productive sprint where we successfully set up the core infrastructure of the project, including the database, Flask application and EC2 deployment which were the most important elements for this sprint. Having these foundations in place early on gave us confidence going into the next sprint. Formalising the Git workflow and pull request process also made collaboration cleaner and reduced the risk of overlapping work as the repository grew in size. Progress was slower than we initially thought, particularly in the first week of the sprint. Some of the estimates in the backlog proved to be optimistic, especially for the database implementation and deployment tasks which took longer than expected. As a result of this we were consistently behind the ideal burndown pace throughout the sprint, but only slightly and our work pace remained fairly consistent. For sprint 3 we aim to be more realistic with our time estimates for sprint tasks, particularly for the tasks that involve integration and deployment.

Show less

00:30 2 / 2

Here's the trimmed version:

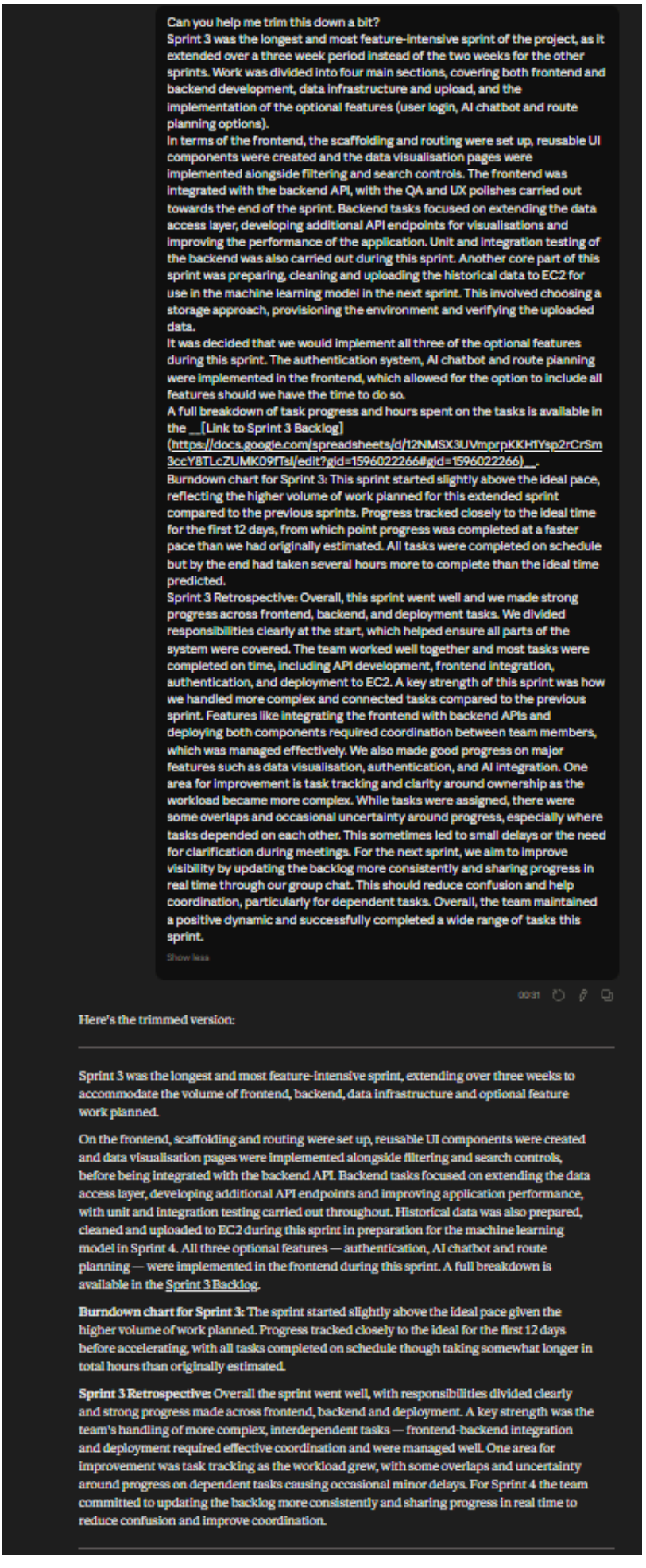
Sprint 2 focused on transitioning from the planning phase into implementation, with primary goals covering database design, the Flask application framework, a preliminary frontend and deployment.

Web scraping was implemented for both bike availability and weather data, with station data collected at five-minute intervals and weather data collected across current, hourly and daily endpoints. A database schema was designed and documented with an ER diagram, with error handling, logging and data retention cleanup processes put in place to ensure reliability. JSON responses from both APIs were parsed and transformed into structured data before being stored in the database. The Flask application skeleton was created with endpoints established for live and stored bike and weather data, with stations rendered on the map using live data for the first time. A Git workflow was also formalised during this sprint, including a pull request template and feature branch process. A preliminary frontend was created using hardcoded data to visualise the general layout ahead of full integration. A full breakdown is available in the Sprint 2 Backlog.

Burndown chart for Sprint 2: The sprint started slightly behind the ideal pace, reflecting the volume of setup and infrastructure work required early on and some optimistic initial estimates. Progress remained consistent throughout and all tasks were completed by the end of the two weeks.

Sprint 2 Retrospective: This was a productive sprint, with the core infrastructure — database, Flask application and EC2 deployment — successfully established. Formalising the Git workflow also improved collaboration as the repository grew. Progress was slower than anticipated in the first week, with database implementation and deployment taking longer than estimated, though the overall pace remained steady. For Sprint 3 the team aimed to make more realistic time estimates, particularly for integration and deployment tasks.

Sprint 3:



Can you help me trim this down a bit?
Sprint 3 was the longest and most feature-intensive sprint of the project, as it extended over a three week period instead of the two weeks for the other sprints. Work was divided into four main sections, covering both frontend and backend development, data infrastructure and upload, and the implementation of the optional features (user login, AI chatbot and route planning options).
In terms of the frontend, the scaffolding and routing were set up, reusable UI components were created and the data visualisation pages were implemented alongside filtering and search controls. The frontend was integrated with the backend API, with the QA and UX polishes carried out towards the end of the sprint. Backend tasks focused on extending the data access layer, developing additional API endpoints for visualisations and improving the performance of the application. Unit and integration testing of the backend was also carried out during this sprint. Another core part of this sprint was preparing, cleaning and uploading the historical data to EC2 for use in the machine learning model in the next sprint. This involved choosing a storage approach, provisioning the environment and verifying the uploaded data.
It was decided that we would implement all three of the optional features during this sprint. The authentication system, AI chatbot and route planning were implemented in the frontend, which allowed for the option to include all features should we have the time to do so.
A full breakdown of task progress and hours spent on the tasks is available in the __[Link to Sprint 3 Backlog](https://docs.google.com/spreadsheets/d/12NMSX3UVmprpKKHfYsp2rCrSm3ccY8TLcZUMK09fTsI/edit?gid=1596022266#gid=1596022266)__.
Burndown chart for Sprint 3: This sprint started slightly above the ideal pace, reflecting the higher volume of work planned for this extended sprint compared to the previous sprints. Progress tracked closely to the ideal time for the first 12 days, from which point progress was completed at a faster pace than we had originally estimated. All tasks were completed on schedule but by the end had taken several hours more to complete than the ideal time predicted.
Sprint 3 Retrospective: Overall, this sprint went well and we made strong progress across frontend, backend, and deployment tasks. We divided responsibilities clearly at the start, which helped ensure all parts of the system were covered. The team worked well together and most tasks were completed on time, including API development, frontend integration, authentication, and deployment to EC2. A key strength of this sprint was how we handled more complex and connected tasks compared to the previous sprint. Features like integrating the frontend with backend APIs and deploying both components required coordination between team members, which was managed effectively. We also made good progress on major features such as data visualisation, authentication, and AI integration. One area for improvement is task tracking and clarity around ownership as the workload became more complex. While tasks were assigned, there were some overlaps and occasional uncertainty around progress, especially where tasks depended on each other. This sometimes led to small delays or the need for clarification during meetings. For the next sprint, we aim to improve visibility by updating the backlog more consistently and sharing progress in real time through our group chat. This should reduce confusion and help coordination, particularly for dependent tasks. Overall, the team maintained a positive dynamic and successfully completed a wide range of tasks this sprint.
Show less

00:31

Here's the trimmed version:

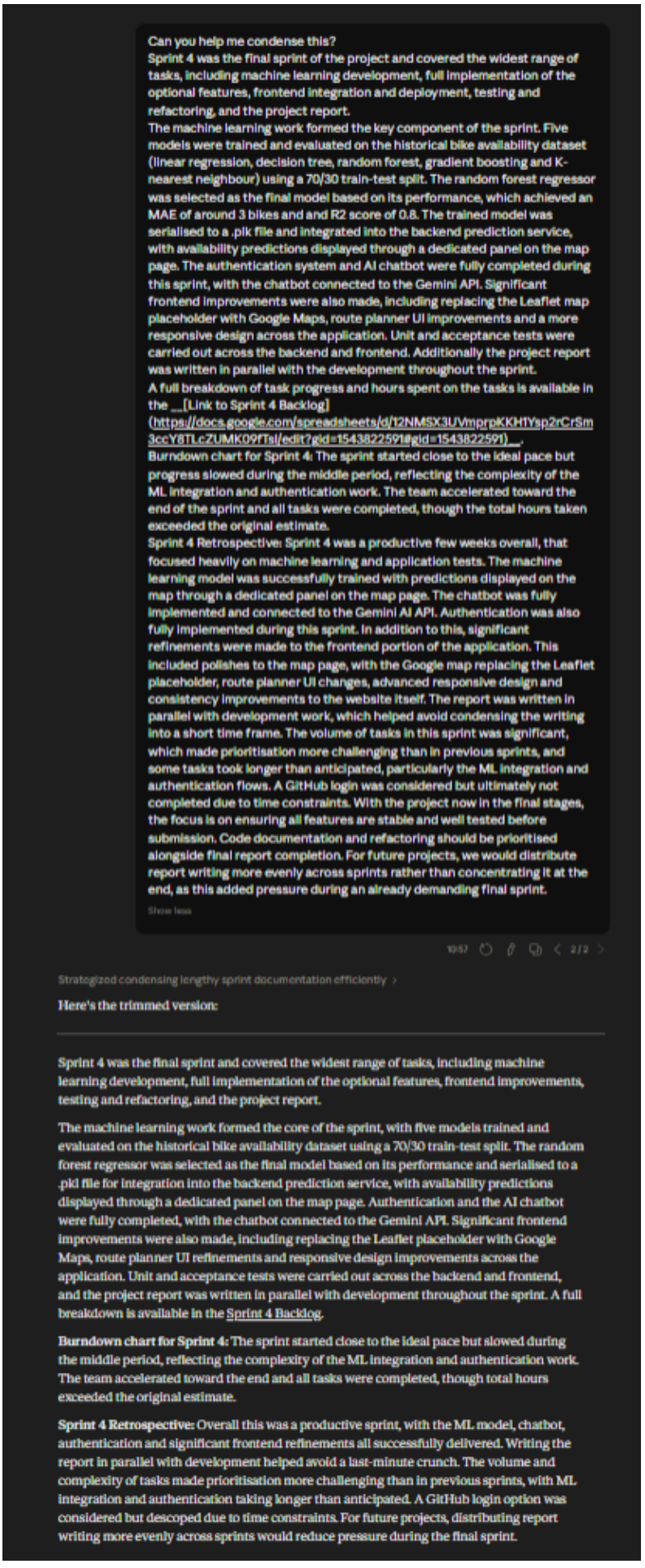
Sprint 3 was the longest and most feature-intensive sprint, extending over three weeks to accommodate the volume of frontend, backend, data infrastructure and optional feature work planned.

On the frontend, scaffolding and routing were set up, reusable UI components were created and data visualisation pages were implemented alongside filtering and search controls, before being integrated with the backend API. Backend tasks focused on extending the data access layer, developing additional API endpoints and improving application performance, with unit and integration testing carried out throughout. Historical data was also prepared, cleaned and uploaded to EC2 during this sprint in preparation for the machine learning model in Sprint 4. All three optional features — authentication, AI chatbot and route planning — were implemented in the frontend during this sprint. A full breakdown is available in the Sprint 3 Backlog.

Burndown chart for Sprint 3: The sprint started slightly above the ideal pace given the higher volume of work planned. Progress tracked closely to the ideal for the first 12 days before accelerating, with all tasks completed on schedule though taking somewhat longer in total hours than originally estimated.

Sprint 3 Retrospective: Overall the sprint went well, with responsibilities divided clearly and strong progress made across frontend, backend and deployment. A key strength was the team's handling of more complex, interdependent tasks — frontend-backend integration and deployment required effective coordination and were managed well. One area for improvement was task tracking as the workload grew, with some overlaps and uncertainty around progress on dependent tasks causing occasional minor delays. For Sprint 4 the team committed to updating the backlog more consistently and sharing progress in real time to reduce confusion and improve coordination.

Sprint 4:



Can you help me condense this?
Sprint 4 was the final sprint of the project and covered the widest range of tasks, including machine learning development, full implementation of the optional features, frontend integration and deployment, testing and refactoring, and the project report.
The machine learning work formed the key component of the sprint. Five models were trained and evaluated on the historical bike availability dataset (linear regression, decision tree, random forest, gradient boosting and K-nearest neighbour) using a 70/30 train-test split. The random forest regressor was selected as the final model based on its performance, which achieved an MAE of around 3 bikes and and R2 score of 0.8. The trained model was serialised to a .pik file and integrated into the backend prediction service, with availability predictions displayed through a dedicated panel on the map page. The authentication system and AI chatbot were fully completed during this sprint, with the chatbot connected to the Gemini API. Significant frontend improvements were also made, including replacing the Leaflet map placeholder with Google Maps, route planner UI improvements and a more responsive design across the application. Unit and acceptance tests were carried out across the backend and frontend. Additionally the project report was written in parallel with the development throughout the sprint.
A full breakdown of task progress and hours spent on the tasks is available in the __[Link to Sprint 4 Backlog](https://docs.google.com/spreadsheets/d/12NMSX3UVmprpKKH1Ysp2rCrSm3ccY8TLcZUMK09fTsI/edit?gid=1543822591#gid=1543822591)__.
Burndown chart for Sprint 4: The sprint started close to the ideal pace but progress slowed during the middle period, reflecting the complexity of the ML integration and authentication work. The team accelerated toward the end of the sprint and all tasks were completed, though the total hours taken exceeded the original estimate.
Sprint 4 Retrospective: Sprint 4 was a productive few weeks overall, that focused heavily on machine learning and application tests. The machine learning model was successfully trained with predictions displayed on the map through a dedicated panel on the map page. The chatbot was fully implemented and connected to the Gemini AI API. Authentication was also fully implemented during this sprint. In addition to this, significant refinements were made to the frontend portion of the application. This included polishes to the map page, with the Google map replacing the Leaflet placeholder, route planner UI changes, advanced responsive design and consistency improvements to the website itself. The report was written in parallel with development work, which helped avoid condensing the writing into a short time frame. The volume of tasks in this sprint was significant, which made prioritisation more challenging than in previous sprints, and some tasks took longer than anticipated, particularly the ML integration and authentication flows. A GitHub login was considered but ultimately not completed due to time constraints. With the project now in the final stages, the focus is on ensuring all features are stable and well tested before submission. Code documentation and refactoring should be prioritised alongside final report completion. For future projects, we would distribute report writing more evenly across sprints rather than concentrating it at the end, as this added pressure during an already demanding final sprint.
Show less

10:57 2/2

Strategized condensing lengthy sprint documentation efficiently

Here's the trimmed version:

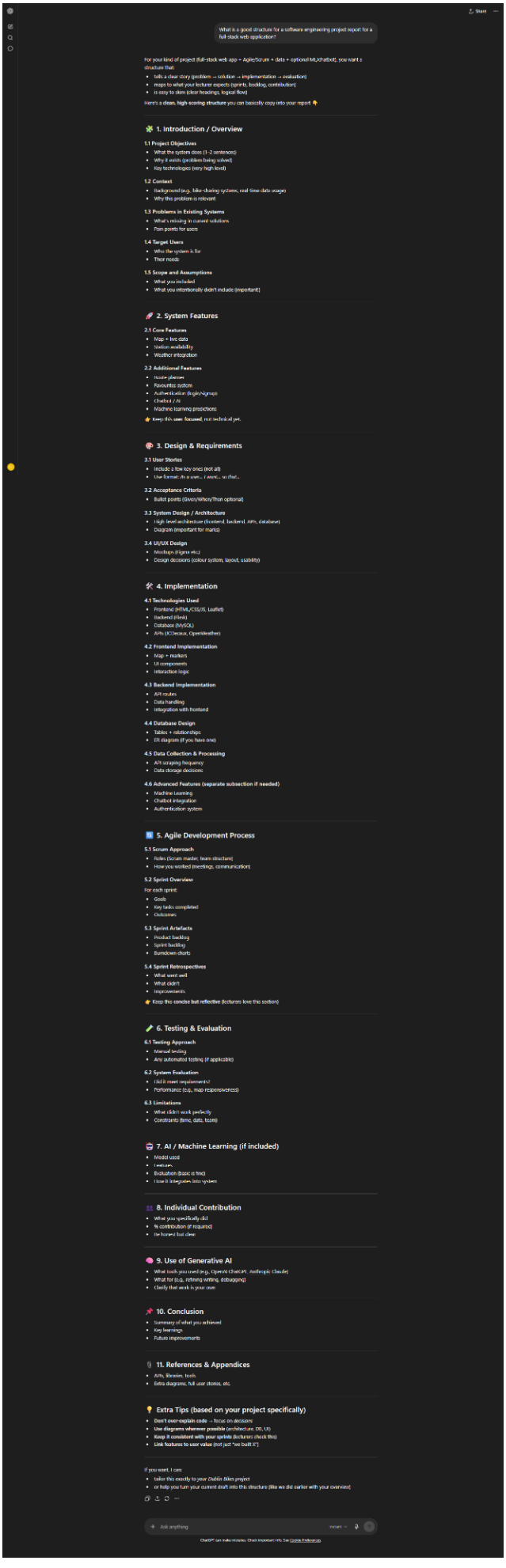
Sprint 4 was the final sprint and covered the widest range of tasks, including machine learning development, full implementation of the optional features, frontend improvements, testing and refactoring, and the project report.

The machine learning work formed the core of the sprint, with five models trained and evaluated on the historical bike availability dataset using a 70/30 train-test split. The random forest regressor was selected as the final model based on its performance and serialised to a .pkl file for integration into the backend prediction service, with availability predictions displayed through a dedicated panel on the map page. Authentication and the AI chatbot were fully completed, with the chatbot connected to the Gemini API. Significant frontend improvements were also made, including replacing the Leaflet placeholder with Google Maps, route planner UI refinements and responsive design improvements across the application. Unit and acceptance tests were carried out across the backend and frontend, and the project report was written in parallel with development throughout the sprint. A full breakdown is available in the Sprint 4 Backlog.

Burndown chart for Sprint 4: The sprint started close to the ideal pace but slowed during the middle period, reflecting the complexity of the ML integration and authentication work. The team accelerated toward the end and all tasks were completed, though total hours exceeded the original estimate.

Sprint 4 Retrospective: Overall this was a productive sprint, with the ML model, chatbot, authentication and significant frontend refinements all successfully delivered. Writing the report in parallel with development helped avoid a last-minute crunch. The volume and complexity of tasks made prioritisation more challenging than in previous sprints, with ML integration and authentication taking longer than anticipated. A GitHub login option was considered but descoped due to time constraints. For future projects, distributing report writing more evenly across sprints would reduce pressure during the final sprint.

Report Structure:



Share

What is a good structure for a software engineering project report for a full-stack web application?

For your kind of project (full-stack web app + Agile/Scrum + data + optional ML/chatbot), you want a structure that:

- tells a clear story (problem → solution → implementation → evaluation)
- maps to what your lecturer expects (sprints, backlog, contribution)
- is easy to skim (clear headings, logical flow)

Here's a clean, high-scoring structure you can basically copy into your report 👇

🌟 1. Introduction / Overview

1.1 Project Objectives
- What the system does (1–2 sentences)
- Why it exists (problem being solved)
- Key technologies (very high level)

1.2 Context
- Background (e.g., bike-sharing systems, real-time data usage)
- Why this problem is relevant

1.3 Problems in Existing Systems
- What's missing in current solutions
- Pain points for users

1.4 Target Users
- Who the system is for
- Their needs

1.5 Scope and Assumptions
- What you included
- What you intentionally didn't include (important!)

🚀 2. System Features

2.1 Core Features
- Map + live data
- Station availability
- Weather integration

2.2 Additional Features
- Route planner
- Favourites system
- Authentication (login/signup)
- Chatbot / AI
- Machine learning predictions

👉 Keep this **user-focused**, not technical yet.

🧠 3. Design & Requirements

3.1 User Stories
- Include a few key ones (not all)
- Use format: *As a user... I want... so that...*

3.2 Acceptance Criteria
- Bullet points (Given/When/Then optional)

3.3 System Design / Architecture
- High-level architecture (frontend, backend, APIs, database)
- Diagram (important for marks)

3.4 UI/UX Design
- Mockups (Figma etc.)
- Design decisions (colour system, layout, usability)

🛠️ 4. Implementation

4.1 Technologies Used
- Frontend (HTML/CSS/JS, Leaflet)
- Backend (Flask)
- Database (MySQL)
- APIs (JCDecaux, OpenWeather)

4.2 Frontend Implementation
- Map + markers
- UI components
- Interaction logic

4.3 Backend Implementation
- API routes
- Data handling
- Integration with frontend

4.4 Database Design
- Tables + relationships
- ER diagram (if you have one)

4.5 Data Collection & Processing
- API scraping frequency
- Data storage decisions

4.6 Advanced Features (separate subsection if needed)
- Machine Learning
- Chatbot integration
- Authentication system

🔁 5. Agile Development Process

5.1 Scrum Approach
- Roles (Scrum master, team structure)
- How you worked (meetings, communication)

5.2 Sprint Overview

For each sprint:
- Goals
- Key tasks completed
- Outcomes

5.3 Sprint Artefacts
- Product backlog
- Sprint backlog
- Burndown charts

5.4 Sprint Retrospectives
- What went well
- What didn't
- Improvements

👉 Keep this concise but reflective (lecturers love this section)

🧪 6. Testing & Evaluation

6.1 Testing Approach
- Manual testing
- Any automated testing (if applicable)

6.2 System Evaluation
- Did it meet requirements?
- Performance (e.g., map responsiveness)

6.3 Limitations
- What didn't work perfectly
- Constraints (time, data, team)

🤖 7. AI / Machine Learning (if included)
- Model used
- Features
- Evaluation (basic is fine)
- How it integrates into system

👥 8. Individual Contribution
- What you specifically did
- % contribution (if required)
- Be honest but clear

🧠 9. Use of Generative AI
- What tools you used (e.g., OpenAI ChatGPT, Anthropic Claude)
- What for (e.g., refining writing, debugging)
- Clarify that work is your own

📌 10. Conclusion
- Summary of what you achieved
- Key learnings
- Future improvements

📎 11. References & Appendices
- APIs, libraries, tools
- Extra diagrams, full user stories, etc.

💡 Extra Tips (based on your project specifically)
- **Don't over-explain code** → focus on *decisions*
- **Use diagrams wherever possible** (architecture, DB, UI)
- **Keep it consistent with your sprints** (lecturers check this)
- **Link features to user value** (not just "we built it")

If you want, I can:
- tailor this exactly to *your Dublin Bikes project*
- or help you turn your current draft into this structure (like we did earlier with your overview)

Ask anything

ChatGPT can make mistakes. Check important info. See Cookie Preferences.